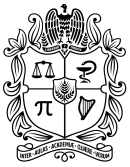
Informe final

Taller No 01 Simulación de MANET

Modelos estocásticos y simulación en computación y comunicaciones

Autores:
David Fernando Benjumea Mora
Daniel Alonso Gracia Pinto
Stephan Alain Roland García Martiquet
José Esteban Alvarado Monroy
Óscar Iván Ulises Gutiérrez Palacios
John Jairo Páez Albino

2026-2S
Profesor: Jorge Eduardo Ortiz Triviño



UNIVERSIDAD
NACIONAL
DE COLOMBIA

4 de noviembre de 2026

Índice

1 Descripción y justificación del problema **3**
1.1 Contexto . . . 3
1.2 El problema: dos niveles de movilidad . . . 3
1.3 Justificación . . . 4
1.4 Alcance . . . 4

2 Pregunta de investigación e hipótesis **4**
2.1 Método científico aplicado . . . 4
2.2 Pregunta de investigación . . . 4
2.3 Hipótesis . . . 4
2.4 Variables . . . 5
2.5 Diseño experimental . . . 6
2.6 Método de análisis estadístico . . . 6

3 Marco teórico **7**
3.1 Redes móviles *ad hoc* (MANET) . . . 8
3.2 Arquitecturas jerárquicas y *clustering* . . . 8
3.3 Antecedente: escalabilidad de redes *ad hoc* jerárquicas . . . 8
3.4 Modelos de movilidad . . . 8
3.5 Enrutamiento: el protocolo AODV . . . 9
3.6 Métricas de desempeño . . . 10
3.7 Simulación de eventos discretos y ns-3 . . . 10
3.8 Análisis estadístico de los resultados . . . 11

4 Diseño de la solución **11**
4.1 Arquitectura jerárquica . . . 12
4.2 Modelo de movilidad jerárquica . . . 12
4.3 Flujo de la simulación . . . 13
4.4 Diseño experimental . . . 15
4.5 Decisiones de diseño y su justificación . . . 15

5 Código fuente **16**
5.1 Entorno de software y versiones . . . 16
5.2 Estructura de `manet-jerarquica.cc` . . . 16
5.3 Generación de tráfico sintético (`ConfigurarTrafico`) . . . 17
5.4 Recolección y reporte de métricas (`FlowMonitor`) . . . 17
5.5 Soporte para NetAnim (`ConfigurarNetAnim`) . . . 18
5.6 Fragmentos clave de la implementación . . . 18
5.6.1 Configuración de flujos CBR . . . 18
5.6.2 Cálculo y guardado de métricas . . . 19
5.7 Validación de integración del programa . . . 19

6 Manual de usuario y manual técnico **20**

7 Experimentación y análisis de resultados **20**
7.1 Escenarios y ejecución . . . 20
7.2 Resultados agregados . . . 20
7.3 Interpretación por escenario . . . 20
7.4 Comparación E1 vs. E2: prueba de la hipótesis principal . . . 21
7.5 Efecto de la velocidad e hipótesis secundarias . . . 22

8 Conclusiones **23**

9 Recomendaciones **24**
9.1 Limitaciones del estudio 24
9.2 Trabajo futuro 25

1 Descripción y justificación del problema

1.1 Contexto

Una red móvil ad hoc (MANET, *Mobile Ad hoc Network*) es un conjunto de nodos inalámbricos que se comunican entre sí sin depender de una infraestructura fija, como puntos de acceso o estaciones base. Por ello resulta especialmente útil donde esa infraestructura no existe o no alcanza, como en zonas rurales sin cobertura o en eventos multitudinarios. En ella cada nodo cumple un doble papel: es terminal y, a la vez, enrutador, de modo que reenvía los paquetes de otros nodos para que la información llegue a destinos situados fuera de su alcance directo [14, 21]. Esta independencia tiene un costo, dado que los nodos se mueven: la topología de la red cambia continuamente, los enlaces aparecen y desaparecen, y las rutas deben descubrirse y repararse sobre la marcha. Dicho de otro modo, la red no es una estructura estática, sino una que se reorganiza mientras transmite.

A medida que crece el número de nodos, una organización plana deja de escalar. En protocolos reactivos como AODV [13], cada descubrimiento de ruta inunda la red con solicitudes, y el tráfico de control termina compitiendo con el tráfico útil. La alternativa habitual es una **arquitectura jerárquica**: los nodos se agrupan en clústeres, cada clúster tiene un nodo coordinador o *cluster head* (CH), y los CH forman a su vez un nivel superior (*backbone*) por el que viaja la comunicación entre clústeres. Ospina y Ortiz [12] estudiaron, con simulaciones en ns-3, las condiciones bajo las cuales estas arquitecturas de dos y tres capas son escalables, y mostraron que la relación de recursos entre capas sucesivas puede describirse con un factor de crecimiento. La jerarquía, en suma, ordena la red, pero también introduce enlaces de los que depende buena parte del tráfico.

1.2 El problema: dos niveles de movilidad

En una MANET jerárquica la movilidad no ocurre de una sola forma, sino en dos niveles distintos:

- **Movilidad de clúster:** el grupo completo se desplaza como un todo. Es el caso de un convoy de vehículos, una brigada de rescate o un pelotón que avanza junto.
- **Movilidad de nodo:** cada integrante se mueve individualmente dentro del área de su grupo, sin alejarse de él.

Los modelos de movilidad clásicos, como Random Waypoint o Random Walk, tratan a cada nodo como independiente. Los modelos de grupo, como RPGM (*Reference Point Group Mobility*), representan en cambio esta combinación de un movimiento colectivo con desviaciones individuales [5, 7].

Ambos tipos de movilidad afectan a la red de forma diferente. La movilidad de nodo cambia muchos enlaces, pero todos son *locales*: rutas cortas dentro de un mismo clúster, que AODV repara con poco costo. La movilidad de clúster, por el contrario, cambia pocos enlaces, pero son los enlaces *críticos* entre los CH, por donde pasa todo el tráfico que cruza la jerarquía; cuando uno de ellos se rompe, se interrumpen todos los flujos entre clústeres que dependían de él. Se trata, entonces, de muchos daños pequeños frente a pocos daños graves, y no es evidente cuál de los dos efectos domina ni cómo cambia esa relación al aumentar la velocidad.

El problema que aborda este trabajo es, por tanto, **cuantificar y comparar, mediante simulación en ns-3, el efecto de la movilidad de clúster y el de la movilidad de nodo sobre el desempeño de una MANET jerárquica** de dos niveles (cuatro clústeres en el primer nivel y dos superclústeres en el segundo), con enrutamiento AODV.

1.3 Justificación

La pregunta no es solo de interés académico; responderla tiene consecuencias en al menos cuatro frentes:

- **Diseño de redes jerárquicas.** Saber qué tipo de movilidad degrada más la red orienta las decisiones de diseño. Si domina la movilidad de clúster, conviene invertir en la estabilidad del backbone (elección de CH estables, gateways redundantes); si domina la de nodo, conviene invertir en el mantenimiento de rutas dentro de cada clúster.
- **Escenarios reales con movimiento en grupo.** Operaciones de rescate, redes militares tácticas, caravanas vehiculares y redes de sensores en animales presentan exactamente esta combinación de movimiento colectivo e individual.
- **Pertinencia de la simulación.** Montar un experimento de campo con decenas de equipos móviles es costoso y difícil de repetir. La simulación, en cambio, permite activar y desactivar cada tipo de movilidad por separado —algo imposible de controlar en el mundo real—, mantener constantes las demás variables y repetir cada experimento con distintas semillas aleatorias.
- **Relación con el curso.** La movilidad es un proceso estocástico, y cada corrida de la simulación es una realización de ese proceso. Por eso las conclusiones no se apoyan en una sola corrida, sino en réplicas independientes, estimadores e intervalos de confianza, como se describe en la sección 2.

1.4 Alcance

El estudio incluye una jerarquía fija de 20 nodos (cuatro clústeres de cinco nodos con CH predefinido y dos superclústeres), el protocolo AODV, tráfico UDP de tasa constante y cuatro escenarios de movilidad evaluados a cuatro velocidades. Quedan fuera del alcance la reelección dinámica de CH, la comparación entre protocolos de enrutamiento, el consumo de energía y el uso de la librería ns3-ai. Estos aspectos no se descartan por falta de interés, sino para mantener el experimento acotado, y se proponen como trabajo futuro.

2 Pregunta de investigación e hipótesis

2.1 Método científico aplicado

El trabajo sigue las etapas del método científico [4]. Más que un formalismo, este método obliga a fijar la pregunta, la hipótesis y el criterio de decisión *antes* de mirar los datos, lo que evita acomodar las conclusiones a los resultados. La tabla 1 resume cómo se concreta cada etapa.

2.2 Pregunta de investigación

> *En una MANET jerárquica de dos niveles con enrutamiento AODV, ¿qué degrada más el desempeño de la red —la movilidad a nivel de clúster o la movilidad a nivel de nodo— y cómo cambia ese efecto al aumentar la velocidad?*

2.3 Hipótesis

Hipótesis principal. A una misma velocidad, la movilidad de clúster degrada más el desempeño que la movilidad de nodo: produce una menor tasa de entrega de paquetes y un mayor retardo. La razón esperada es que el movimiento de un clúster rompe los enlaces entre cluster heads, por los que pasa todo el tráfico entre clústeres, mientras que el movimiento

Etapa	En este trabajo
Observación	En una MANET jerárquica la movilidad ocurre en dos niveles (clúster y nodo), que afectan enlaces de naturaleza distinta (backbone y enlaces locales).
Pregunta	¿Cuál de los dos niveles de movilidad degrada más el desempeño, y cómo depende de la velocidad? (sección 2.2)
Hipótesis	La movilidad de clúster degrada más que la de nodo (sección 2.3).
Predicción	Con la misma velocidad, la PDR del escenario con solo movilidad de clúster será menor y su retardo mayor que en el escenario con solo movilidad de nodo.
Experimento	Simulación controlada en ns-3 con cuatro escenarios, cuatro velocidades y diez réplicas por configuración (sección 2.5).
Análisis	Medias, intervalos de confianza del 95 % y pruebas t de Welch (sección 2.6).
Conclusión	Se acepta o se rechaza la hipótesis según la regla de decisión definida de antemano.

Cuadro 1: Etapas del método científico aplicadas al estudio.

de los nodos solo altera rutas cortas dentro de cada clúster. Se apuesta, en otras palabras, a que importa más *qué* enlace se rompe que *cuántos* se rompen.

Sea $\mu_k^{\mathrm{PDR}}(v)$ la PDR media del escenario k a la velocidad v, y $\mu_k^{D}(v)$ su retardo medio. Para cada velocidad v se plantean:

$$H_0^{\mathrm{PDR}} : \mu_{E2}^{\mathrm{PDR}}(v) = \mu_{E1}^{\mathrm{PDR}}(v) \qquad H_1^{\mathrm{PDR}} : \mu_{E2}^{\mathrm{PDR}}(v) < \mu_{E1}^{\mathrm{PDR}}(v)$$
$$H_0^{D} : \mu_{E2}^{D}(v) = \mu_{E1}^{D}(v) \qquad H_1^{D} : \mu_{E2}^{D}(v) > \mu_{E1}^{D}(v)$$

donde E1 es el escenario con solo movilidad de nodo y E2 el escenario con solo movilidad de clúster (tabla 3).

Hipótesis secundarias. La hipótesis principal compara los dos tipos de movilidad a una velocidad dada; las secundarias amplían la mirada al papel de la velocidad y a la combinación de ambos movimientos:

- **HS1:** en los escenarios con movimiento (E1, E2 y E3), el desempeño empeora a medida que aumenta la velocidad.
- **HS2:** el escenario combinado E3 tiene un desempeño igual o peor que el peor de E1 y E2.
- **HS3:** la diferencia entre E1 y E2 crece con la velocidad.

2.4 Variables

Tipo	Variables
Independientes	Tipo de movilidad (ninguna, nodo, clúster, ambas) y velocidad $v \in \{1, 5, 10, 20\}$ m/s.
Dependientes	PDR, retardo extremo a extremo medio, throughput y jitter medio.
Controladas	Número y agrupación de nodos, geometría, estándar WiFi, tasa de transmisión, potencia, modelo de propagación, protocolo de enrutamiento, tráfico y tiempo de simulación (tabla 4).

Cuadro 2: Variables del experimento.

Las métricas se calculan a partir de los contadores que entrega FlowMonitor (`txPackets`, `rxPackets`, `rxBytes`, `delaySum`, `jitterSum`, `timeFirstTxPacket`, `timeLastRxPacket`), sumando sobre todos los flujos f de una corrida:

$$\text{PDR} = \frac{\sum_f \text{rxPackets}_f}{\sum_f \text{txPackets}_f}, \qquad \bar{D} = \frac{\sum_f \text{delaySum}_f}{\sum_f \text{rxPackets}_f},$$

$$\text{Throughput} = \frac{8 \sum_f \text{rxBytes}_f}{t_{\text{fin}} - t_{\text{inicio}}}, \qquad \bar{J} = \frac{\sum_f \text{jitterSum}_f}{\sum_f (\text{rxPackets}_f - 1)},$$

donde t_{inicio} y t_{fin} son el inicio y el fin del periodo de tráfico.

2.5 Diseño experimental

Escenario	Clúster	Nodo	Propósito
E0	No	No	Control (línea base sin movilidad).
E1	No	Sí	Efecto aislado de la movilidad de nodo.
E2	Sí	No	Efecto aislado de la movilidad de clúster.
E3	Sí	Sí	Efecto combinado de ambos niveles.

Cuadro 3: Escenarios de movilidad.

E0 cumple el papel de control: muestra cómo se comporta la red cuando nada se mueve y sirve de referencia para medir la degradación de los demás escenarios. Los escenarios E1, E2 y E3 se evalúan a las cuatro velocidades, mientras que E0 se evalúa una sola vez porque no tiene movimiento. Cada configuración se repite $n = 10$ veces con semillas distintas (parámetro `RngRun` de ns-3), lo que da $(3 \times 4 + 1) \times 10 = 130$ corridas.

Para que E1 y E2 sean comparables, en ambos se mueve **una sola capa** a la misma velocidad v: los nodos en E1 y los clústeres en E2. Así, cualquier diferencia entre ellos puede atribuirse al tipo de movilidad y no a otra variable. Los superclústeres permanecen fijos en todos los escenarios; su función es organizar la jerarquía y delimitar la región en la que se mueven sus clústeres.

El alcance de cada canal se fija explícitamente con un modelo de disco (`RangePropagationLossModel`): todo receptor dentro del alcance recibe la señal y ninguno fuera de él. Con el modelo por defecto de ns-3 (pérdida log-distancia), la potencia de transmisión por defecto y el umbral de detección de preámbulo de ns-3.43 ($-82\,\text{dBm}$), un enlace 802.11b a 1 Mbps solo alcanza unos 50 m (medido: 10 de 10 paquetes a 50 m y ninguno a 55 m), lo que impediría tener un backbone entre clústeres separados 150 m. El modelo de disco, además, hace que la geometría del experimento sea fácil de controlar y de explicar. Un alcance de 60 m dentro del clúster (comparable a la región de ± 40 m en que se mueven los nodos) y de 250 m entre CH (comparable a la separación de 150 m entre clústeres y a su región de ± 150 m) hace que ambos tipos de movilidad puedan romper enlaces, que es justamente lo que se quiere medir. Se renuncia con ello a algo de realismo físico a cambio de un experimento cuyo comportamiento se puede razonar y verificar.

2.6 Método de análisis estadístico

Cada corrida es una realización independiente del proceso estocástico de movilidad, de modo que una sola corrida no basta para concluir nada: dos semillas distintas pueden dar resultados muy diferentes. Por eso, para cada configuración y cada métrica se calcula la media muestral $\bar{x}$, la desviación estándar s y el intervalo de confianza del 95 %:

$$\bar{x} \pm t_{0.975,\, n-1} \, \frac{s}{\sqrt{n}}, \qquad t_{0.975,\, 9} \approx 2.262.$$

Parámetro	Valor
Simulador	ns-3.43 (entorno de referencia: Ubuntu 22.04; también ejecutable en Windows nativo u otras distribuciones de Linux)
Nodos	20: 4 clústeres de 5 nodos (1 CH fijo + 4 miembros)
Jerarquía	Nivel 1: clústeres A, B, C, D; nivel 2: X = {A, B}, Y = {C, D}
Canales	Un canal WiFi por clúster y un canal backbone compartido por los 4 CH
Estándar WiFi	IEEE 802.11b, modo ad hoc, tasa fija DSSS 1 Mbps (`ConstantRateWifiManager`)
Propagación	Modelo de disco (`RangePropagationLossModel`) con retardo de velocidad constante
Alcance efectivo	60 m dentro de cada clúster y 250 m en el backbone entre CH
Potencia de transmisión	Valor por defecto de ns-3
Enrutamiento	AODV con parámetros por defecto
Tráfico	8 flujos UDP `OnOff` de tasa constante, paquetes de 512 B a 4 paquetes/s (16 kbit/s por flujo)
Pares de tráfico	Dentro de cada superclúster: A→B, B→A, C→D, D→C; entre superclústeres: A→C, B→D, C→A, D→B (miembro a miembro)
Tiempos	Simulación de 200 s; tráfico desde un instante uniforme en $[10, 11]$ s hasta 195 s
Movilidad de clúster	Random Waypoint relativo al centro de su superclúster, región de ± 150 m, velocidad constante v, pausa de 2 s
Movilidad de nodo	Random Walk 2D relativo al centro de su clúster, región de ± 40 m, velocidad constante v
Posiciones iniciales	Superclústeres en $(350, 300)$ y $(650, 300)$ m; clústeres en $x = 275, 425, 575, 725$ m sobre $y = 300$ m; CH en el centro y miembros en un anillo de 25 m de radio

Cuadro 4: Parámetros constantes de la simulación. Los valores geométricos son valores de partida: si en E0 la PDR resulta menor al 95 %, se ajustan para garantizar una red conectada y el cambio se documenta.

La hipótesis principal se contrasta, para cada velocidad, con una prueba t de Welch unilateral entre E2 y E1 con nivel de significancia $\alpha = 0.05$. Se define de antemano la siguiente **regla de decisión**: la hipótesis principal se considera respaldada si se rechaza H_0^{PDR} en al menos tres de las cuatro velocidades; si se rechaza en menos, no se acepta. El mismo criterio se aplica al retardo. Fijar esta regla de antemano es lo que da peso a la conclusión: el resultado puede respaldar la hipótesis o refutarla, pero no reinterpretarse a posteriori. Las hipótesis secundarias se evalúan comparando las medias y sus intervalos de confianza en las gráficas de desempeño contra velocidad.

3 Marco teórico

Esta sección reúne los conceptos en los que se apoya el resto del trabajo: qué es una red móvil *ad hoc*, cómo se organiza en clústeres, cómo se modela el movimiento de los clústeres y de los nodos, cómo encuentra rutas AODV, con qué métricas se mide el desempeño, cómo se simula todo esto en ns-3 y cómo se analizan los resultados. Cada apartado responde a una pieza del experimento, de modo que el lector pueda seguir más adelante por qué se tomó cada decisión de diseño.

3.1 Redes móviles *ad hoc* (MANET)

Una red inalámbrica *ad hoc* se forma sin infraestructura fija: no hay puntos de acceso ni estaciones base, y cada nodo funciona a la vez como terminal y como enrutador, reenviando los paquetes de otros nodos. Cuando el origen y el destino no están uno dentro del alcance de radio del otro, el mensaje viaja en varios saltos, de vecino en vecino. Una MANET (*Mobile Ad hoc Network*) es una red *ad hoc* cuyos nodos se mueven, de modo que la topología cambia constantemente: los enlaces aparecen y desaparecen según la distancia entre los nodos [14].

Estas redes se despliegan cuando hace falta un servicio y desaparecen cuando este termina, por lo que se usan en operaciones de rescate, redes militares tácticas, redes vehiculares y lugares donde la infraestructura no existe o dejó de funcionar. Sus desafíos principales son tres: el ancho de banda del canal inalámbrico, que comparten todos los nodos vecinos; la energía de los dispositivos, y la inestabilidad de las rutas causada por la movilidad [14]. Este trabajo se centra en el último.

3.2 Arquitecturas jerárquicas y *clustering*

En una red plana todos los nodos están al mismo nivel y participan por igual en el enrutamiento. Es sencilla, pero, a medida que crece el número de nodos, aumenta el tráfico de control, las rutas se alargan y se vuelven inestables, y la red deja de ser escalable. El *clustering* divide la red en subconjuntos de nodos, los clústeres, para hacerla más escalable [2]. Cada clúster elige una cabeza de clúster (*cluster head*, CH) que conoce a sus miembros y sabe cómo alcanzarlos. Los miembros envían su tráfico a su CH, los CH reenvían paquetes en nombre de sus miembros y, entre ellos, ejecutan el protocolo de enrutamiento de la red [2]. Los nodos que conectan clústeres vecinos se llaman pasarelas (*gateways*) [14]. Como los CH pueden agruparse a su vez en clústeres de un nivel superior, se obtiene una jerarquía de varios niveles; en este trabajo los CH se comunican por un canal propio, el *backbone*, y los clústeres se agrupan en dos superclústeres (sección 2).

Los clústeres pueden ser físicos o lógicos. Un clúster físico es un grupo que se desplaza en bloque porque comparte un medio, como los pasajeros de un bus o un convoy de vehículos. Un clúster lógico agrupa nodos por un interés común, como los integrantes de un grupo de trabajo. En ambos casos hay dos movimientos distintos: el del clúster como un todo y el de cada nodo dentro de él, y el modelo de movilidad debe representarlos por separado.

3.3 Antecedente: escalabilidad de redes *ad hoc* jerárquicas

Ospina y Ortiz [12] estudiaron qué condiciones permiten que una red *ad hoc* jerárquica escale según los recursos disponibles en cada capa. Modelaron la disponibilidad de recursos con una distribución geométrica truncada, definieron un factor de crecimiento j que relaciona los recursos de dos capas sucesivas y lo estimaron con simulaciones en ns-3 de arquitecturas de dos y tres capas. Encontraron que el crecimiento de los recursos entre capas sigue un modelo lineal y que el valor estimado de j está relacionado con la razón áurea. Su trabajo es un ejemplo de la misma forma de indagación que sigue este taller: una pregunta sobre una red jerárquica, un modelo estocástico y la simulación en ns-3 como experimento.

3.4 Modelos de movilidad

Un modelo de movilidad describe cómo cambian en el tiempo la posición, la velocidad y la dirección de los nodos. Los resultados de una simulación de MANET dependen mucho del modelo elegido, así que este debe parecerse al comportamiento que se quiere estudiar [5]: un modelo poco realista puede llevar a conclusiones que no se sostienen fuera del simulador. Los modelos se dividen en modelos de entidad, en los que cada nodo se mueve de forma independiente, y modelos de grupo, en los que el movimiento de un nodo depende del de su grupo [5].

***Random Walk* (caminata aleatoria).** Cada nodo escoge una dirección uniforme en $[0, 2\pi)$ y una velocidad uniforme en $[v_{\min}, v_{\max}]$, se mueve así durante un tiempo o una distancia fijos y vuelve a escoger; al llegar al borde del área, rebota. El modelo no tiene memoria: el siguiente movimiento no depende del anterior, lo que produce cambios bruscos de dirección [5].

***Random Waypoint* (punto de paso aleatorio).** Cada nodo escoge un destino uniforme dentro del área y una velocidad uniforme en $[v_{\min}, v_{\max}]$, viaja en línea recta hasta el destino, se detiene durante un tiempo de pausa y repite el proceso. Es el modelo más usado para evaluar protocolos de MANET desde el trabajo de Broch et al. [3]. Tiene dos defectos conocidos. El primero es que los nodos tienden a concentrarse en el centro del área [5]. El segundo es que, si $v_{\min} = 0$, la velocidad promedio decae con el tiempo, porque los nodos que escogen velocidades muy bajas pasan mucho tiempo en tránsito; por eso Yoon et al. [20] recomiendan una velocidad mínima positiva. Además, en lugar de descartar el periodo inicial de la simulación, se puede arrancar desde la distribución estacionaria del modelo [11], que ns-3 ofrece como `SteadyStateRandomWaypointMobilityModel` [16]. Las pausas también importan: mientras un nodo está detenido no se mueve, así que la velocidad media efectiva es menor que la velocidad configurada.

Gauss-Markov. La velocidad y la dirección de cada nodo se actualizan en intervalos fijos y conservan memoria del valor anterior. Para la velocidad,

$$s_n = \alpha\, s_{n-1} + (1-\alpha)\,\bar{s} + \sqrt{1-\alpha^2}\; x_{n-1},$$

donde $\bar{s}$ es la velocidad media, x_{n-1} es una variable aleatoria normal y $\alpha \in [0, 1]$ controla la aleatoriedad: con $\alpha = 0$ el movimiento es completamente aleatorio y con $\alpha = 1$ es lineal. La dirección se actualiza con una ecuación análoga [5, 10]. El modelo evita las paradas y los giros bruscos de los dos anteriores.

Movilidad de grupo por punto de referencia (RPGM). El modelo *Reference Point Group Mobility*, propuesto por Hong et al. [7], asigna a cada grupo un centro lógico, o punto de referencia, que se mueve según un modelo de entidad, por ejemplo *Random Waypoint*. La posición de cada nodo es la del punto de referencia más un desplazamiento aleatorio dentro de un radio. Así el grupo se desplaza en bloque y, al mismo tiempo, cada nodo se mueve alrededor de su lugar en el grupo.

Movilidad jerárquica. La idea de RPGM se puede aplicar en varios niveles. Si $p_S(t)$ es la posición del centro de un superclúster, $p_C(t)$ la posición del centro de un clúster relativa a su superclúster y $p_N(t)$ la posición de un nodo relativa a su clúster, la posición absoluta del nodo es

$$p(t) = p_S(t) + p_C(t) + p_N(t).$$

Cada término puede seguir su propio modelo y su propia velocidad. Por eso este enfoque representa a la vez el movimiento del clúster y el del nodo, y permite activarlos por separado, que es justamente lo que comparan los escenarios E1 y E2. ns-3 implementa esta composición con `HierarchicalMobilityModel`, en la que la posición final es la suma vectorial de la posición del modelo padre y la del modelo hijo, y ofrece `GroupMobilityHelper` para configurar grupos completos según RPGM [16].

3.5 Enrutamiento: el protocolo AODV

Como la topología cambia, los protocolos de enrutamiento de las redes cableadas no son adecuados y se usan protocolos diseñados para redes *ad hoc*. Los proactivos mantienen en todo momento rutas hacia todos los destinos mediante mensajes de control periódicos, como

OLSR [6]; los reactivos buscan una ruta solo cuando hay datos que enviar, y los híbridos combinan ambos enfoques [2]. Los proactivos tienen las rutas listas de inmediato, pero gastan ancho de banda aunque no haya tráfico; los reactivos ahorran ese tráfico de control, pero agregan una demora mientras descubren la ruta. Ninguno es mejor en términos absolutos: la elección depende de cuánto cambia la red y de cuánto tráfico transporta.

AODV (*Ad hoc On-Demand Distance Vector*) es un protocolo reactivo [13]. Cuando un nodo necesita una ruta, difunde una solicitud de ruta (RREQ) que los vecinos retransmiten. El destino, o un nodo que conoce una ruta suficientemente reciente hacia él, contesta con una respuesta de ruta (RREP) que regresa por el camino inverso y deja instalada la ruta en cada salto. Si un enlace activo se rompe, el nodo que lo detecta envía un mensaje de error (RERR) a los nodos que usaban esa ruta, y el origen debe descubrir una nueva. Los números de secuencia de destino evitan los ciclos y distinguen las rutas recientes de las viejas [13]. Por eso la movilidad afecta directamente a AODV: cada enlace roto se traduce en paquetes perdidos o retrasados mientras se encuentra una ruta nueva, y un enlace roto en el *backbone* afecta a todos los flujos que lo atraviesan.

3.6 Métricas de desempeño

El desempeño de la red se mide con cuatro métricas por flujo, agregadas sobre todos los flujos de datos de una corrida (las fórmulas exactas están en la sección 2):

- **Tasa de entrega de paquetes (PDR, *Packet Delivery Ratio*).** Es la proporción de paquetes de datos que llegan a su destino respecto del total enviado por las aplicaciones [3]. Mide la confiabilidad de la red.
- **Retardo extremo a extremo.** Es el tiempo promedio entre el envío y la recepción de los paquetes que sí llegan. Incluye la espera en las colas, las retransmisiones de la capa MAC y el tiempo de descubrimiento de rutas.
- ***Throughput.*** Es la cantidad de bits recibidos por unidad de tiempo durante el periodo de tráfico. Con una carga fija, baja cuando se pierden paquetes.
- ***Jitter.*** Es la variación del retardo entre paquetes consecutivos de un mismo flujo. Un *jitter* alto indica que las rutas cambian o que las colas crecen y se vacían de forma irregular.

En ns-3 las cuatro se calculan con el módulo `flow-monitor`, que registra para cada flujo, entre otros contadores, los paquetes y bytes transmitidos y recibidos (`txPackets`, `rxPackets`, `rxBytes`), la suma de los retardos extremo a extremo de los paquetes recibidos (`delaySum`) y la suma de las variaciones del retardo (`jitterSum`) [15].

3.7 Simulación de eventos discretos y ns-3

Una simulación de eventos discretos representa un sistema cuyo estado cambia solo en instantes puntuales, los eventos, como el envío o la llegada de un paquete. El simulador mantiene una lista de eventos ordenada por tiempo, toma el siguiente, actualiza el estado y programa los eventos que este genera, sin recorrer los instantes intermedios [1]. Un estudio de simulación sigue estas etapas: formulación del problema y definición de objetivos, construcción del modelo conceptual, recolección de datos, programación, verificación y validación del modelo, diseño de experimentos, corridas, análisis de resultados y documentación [1].

ns-3 es un simulador de redes basado en eventos discretos, libre y de código abierto, escrito en C++ y orientado a la investigación y la docencia [21]. Un programa de ns-3 crea nodos (`Node`), les instala dispositivos de red (`NetDevice`) conectados a un canal (`Channel`), la pila de protocolos de Internet con el protocolo de enrutamiento escogido, un modelo de movilidad y aplicaciones que generan tráfico; las clases auxiliares (*helpers*) simplifican cada uno de estos pasos [21]. Los módulos que usa este trabajo son [18]:

- `wifi`: capas física y MAC de IEEE 802.11, con dispositivos en modo *ad hoc* y canales independientes (`YansWifiChannel`).
- `aodv`: el protocolo AODV descrito en la sección 3.5.
- `mobility`: los modelos de movilidad, entre ellos `RandomWaypointMobilityModel`, `RandomWalk2dMobilityModel` y `HierarchicalMobilityModel`.
- `internet` y `applications`: la pila TCP/IP y las aplicaciones de tráfico, como `OnOffApplication`.
- `flow-monitor`: las métricas por flujo de la sección 3.6.

El alcance de cada canal se fija con un modelo de disco (`RangePropagationLossModel`): dos nodos tienen enlace si y solo si su distancia no supera el alcance. Es la misma idealización que el grafo de disco unitario con que se analizan teóricamente las redes *ad hoc* [2].

Las variables aleatorias de ns-3 provienen del generador MRG32k3a de L'Ecuyer [8], en una implementación que ofrece una gran cantidad de flujos y subflujos independientes [9]. La semilla y el número de corrida (`RngRun`) se fijan con la clase `RngSeedManager`; para obtener réplicas independientes de un mismo experimento, el manual de ns-3 recomienda conservar la semilla y cambiar el número de corrida, que selecciona otro subflujo del generador [17].

3.8 Análisis estadístico de los resultados

El resultado de una corrida es una variable aleatoria, así que una sola corrida no basta para responder la pregunta. Se hacen n réplicas independientes de cada configuración y se estima la media de cada métrica con un intervalo de confianza [1]:

$$\bar{x} \pm t_{1-\alpha/2,\,n-1}\,\frac{s}{\sqrt{n}},$$

donde $\bar{x}$ es la media muestral, s la desviación estándar muestral y $t_{1-\alpha/2,\,n-1}$ el cuantil de la distribución t de Student con $n-1$ grados de libertad.

Para comparar dos escenarios, como E1 y E2, se usa la prueba t de Welch [19], que no supone que las dos poblaciones tengan la misma varianza. Su estadístico y sus grados de libertad aproximados son

$$t = \frac{\bar{x}_2 - \bar{x}_1}{\sqrt{\dfrac{s_1^2}{n_1} + \dfrac{s_2^2}{n_2}}}, \qquad \nu \approx \frac{\left(\dfrac{s_1^2}{n_1} + \dfrac{s_2^2}{n_2}\right)^2}{\dfrac{(s_1^2/n_1)^2}{n_1 - 1} + \dfrac{(s_2^2/n_2)^2}{n_2 - 1}}.$$

La prueba unilateral rechaza la hipótesis nula cuando el valor p es menor que el nivel de significancia α; en este trabajo, $\alpha = 0.05$ (sección 2.6).

4 Diseño de la solución

Esta sección describe cómo se construyó el sistema de simulación con el que se responde la pregunta de investigación. El diseño parte de una restricción metodológica: para poder atribuir los efectos observados a cada tipo de movilidad, el sistema debe permitir activar y desactivar por separado la movilidad de clúster y la de nodo, manteniendo constante todo lo demás. De esa restricción se derivan las cuatro decisiones centrales que se explican a continuación: la topología jerárquica, la movilidad en capas, el flujo interno del programa y el diseño experimental que permite comparar los escenarios.

4.1 Arquitectura jerárquica

La red se organiza en dos niveles de agrupación. En el nivel inferior hay cuatro clústeres, denominados A, B, C y D, cada uno formado por cinco nodos: un *cluster head* (CH) y cuatro miembros. En el nivel superior esos cuatro clústeres se agrupan en dos superclústeres, X = {A, B} y Y = {C, D}. La figura 1 muestra esta estructura.

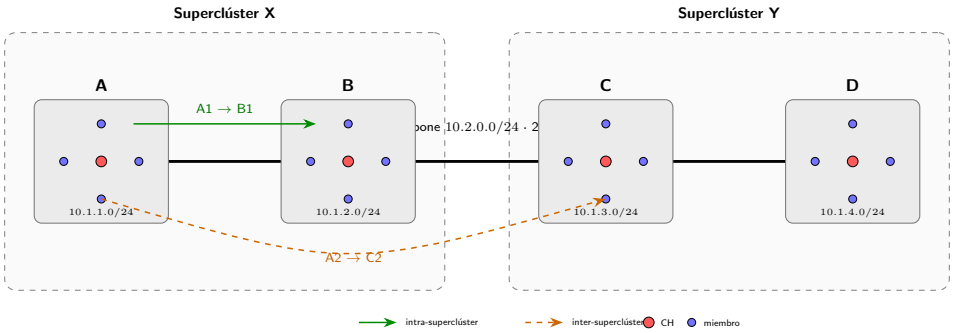


Figura 1: Arquitectura jerárquica de la MANET. Cada clúster tiene un canal WiFi ad hoc independiente y una subred propia; los cuatro *cluster heads* se comunican entre sí por un canal *backbone* compartido.

La comunicación se separa en dos dominios. Dentro de cada clúster, los miembros y su CH comparten un canal WiFi ad hoc independiente, de modo que un miembro solo puede hablar directamente con los nodos de su propio clúster; no hay interferencia ni contienda entre clústeres porque cada uno usa un canal distinto. Para que la información cruce de un clúster a otro, los cuatro CH disponen de una segunda interfaz conectada a un canal *backbone* compartido, por el que se reenvía todo el tráfico interclúster. Esta separación hace que el camino de un paquete entre miembros de clústeres distintos sea siempre de la forma miembro → CH → *backbone* → CH → miembro, lo que convierte a los enlaces entre CH en la columna vertebral de la red.

El direccionamiento refuerza esa estructura: cada clúster es una subred propia (10.1.1.0/24 a 10.1.4.0/24) y el *backbone* es otra (10.2.0.0/24), de manera que el enrutamiento distingue con claridad los saltos locales de los saltos interclúster. Sobre todas las interfaces corre AODV, que al ser un protocolo reactivo descubre las rutas a demanda y las repara cuando un enlace se rompe, exactamente el mecanismo que el experimento quiere poner a prueba.

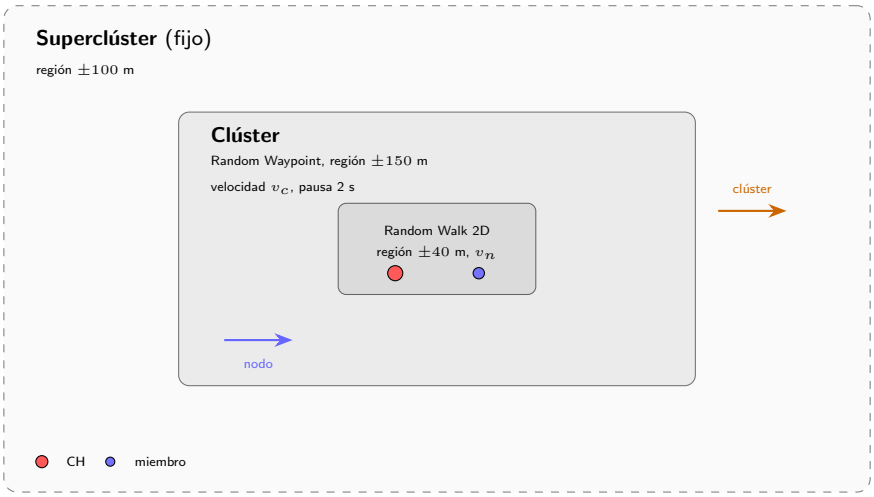
4.2 Modelo de movilidad jerárquica

El movimiento se modela en tres capas anidadas, en correspondencia con los niveles de la jerarquía. Cada capa se desplaza *relativa* a la capa que tiene por encima, de modo que la posición absoluta de un nodo es la suma vectorial de las tres contribuciones:

$$p(t) = p_S(t) + p_C(t) + p_N(t),$$

donde $p_S(t)$ es la posición del centro del superclúster, $p_C(t)$ la del centro del clúster respecto de su superclúster y $p_N(t)$ la del nodo respecto de su clúster. La figura 2 representa esta composición.

Cada capa usa un modelo distinto, acorde con el papel que cumple. Los superclústeres permanecen fijos en todos los experimentos, porque su única función es delimitar la región en la que se mueven sus clústeres; el programa permite moverlos, pero el diseño experimental deja su velocidad en cero. Los clústeres se mueven con un modelo *Random Waypoint* de velocidad constante dentro de una región de ±150 m alrededor del centro de su superclúster,



$$p(t) = p_S(t) + p_C(t) + p_N(t)$$

Figura 2: Composición de la movilidad en tres capas. El clúster se mueve como un todo dentro de su superclúster y, a su vez, cada nodo se mueve dentro de su clúster; la posición final es la suma de las tres contribuciones.

con una pausa fija de 2 s al llegar a cada destino. Los nodos miembros se mueven con una caminata aleatoria bidimensional (*Random Walk 2D*) de velocidad constante dentro de una región de ±40 m alrededor del centro de su clúster, cambiando de rumbo cada 10 m recorridos. El CH permanece en el centro de su clúster. La tabla 5 resume las capas.

Capa	Modelo	Región	Velocidad	Observación
Superclúster	Fijo	±100 m	$v_S = 0$	Solo organiza la jerarquía.
Clúster	Random Waypoint	±150 m	v_C (constante)	Pausa de 2 s entre destinos.
Nodo	Random Walk 2D	±40 m	v_N (constante)	Cambia de rumbo cada 10 m.

Cuadro 5: Capas de movilidad y sus parámetros.

La elección de modelos con velocidad constante es deliberada: en los escenarios E1 y E2 se mueve una sola capa a la misma velocidad v, lo que garantiza que ambos escenarios sean comparables. La región de ±40 m en que se mueven los nodos mantiene a un miembro siempre a menos de 57 m de su CH, dentro del alcance de 60 m del canal del clúster; la movilidad de nodo, por tanto, no puede desconectar a un miembro de su CH, sino que solo altera la calidad de las rutas locales. La movilidad de clúster, en cambio, sí puede alejar a dos CH más allá del alcance de 250 m del *backbone*, que es precisamente el efecto que la hipótesis considera dominante.

4.3 Flujo de la simulación

El programa `manet-jerarquica.cc` sigue una secuencia lineal de fases, cada una implementada en una función independiente. La figura 3 muestra el orden y la correspondencia de

cada fase con la parte del trabajo que la construyó.

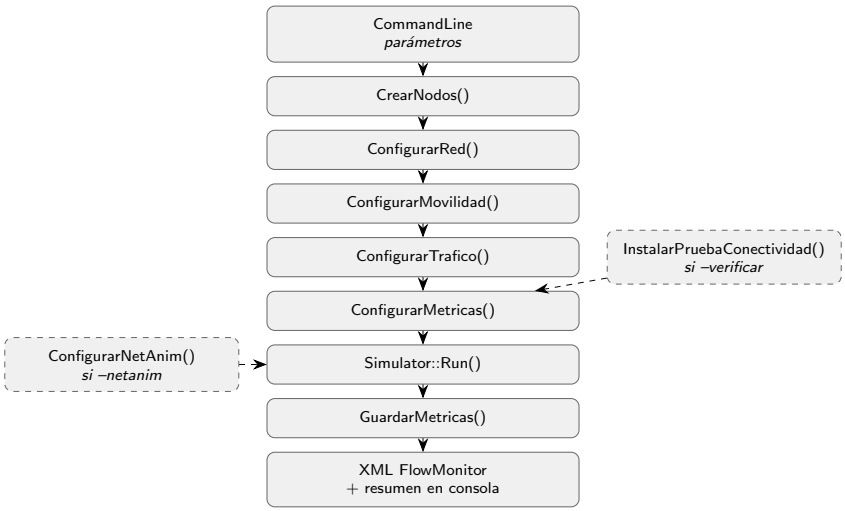


Figura 3: Flujo de la simulación: los parámetros se leen, se crean los nodos y la red, se configuran movilidad, tráfico y métricas, se ejecuta el simulador y se guardan los resultados. Las ramas punteadas corresponden a los modos de verificación y de visualización.

Primero se interpretan los parámetros de línea de comandos, que fijan el escenario, las velocidades, la semilla y el directorio de salida. A continuación se crean los veinte nodos y se los agrupa en clústeres y superclústeres; se instalan las interfaces WiFi con sus canales y la pila IP con AODV; se configura la movilidad jerárquica; se inyectan los ocho flujos de tráfico; y se instala el monitor de métricas. Solo entonces se ejecuta el simulador y, al terminar, se serializan los resultados a XML y se imprime un resumen en consola. Dos modos auxiliares completan el programa: con `--verificar` se corre una prueba de conectividad entre clústeres y con `--netanim` se genera una traza visual para NetAnim.

El algoritmo que resume esa secuencia, independiente de la sintaxis de C++, se presenta a continuación.

```
procedimiento Simular(escenario, v_cluster, v_nodo, semilla):
    leer parámetros y fijar la semilla
    crear 20 nodos
    agrupar en 4 clústeres (A-D) y 2 superclústeres (X, Y)

    para cada clúster:
        crear su canal WiFi propio (alcance 60 m)
        asignar la subred 10.1.k.0/24

    crear el canal backbone (alcance 250 m) para los 4 CH
    instalar la pila IP con AODV

    configurar la movilidad jerárquica:
        superclúster fijo
        clúster con Random Waypoint (±150 m, v_cluster)
        miembro con Random Walk 2D (±40 m, v_nodo)

    instalar 8 flujos UDP OnOff (puertos 5000-5007)
    instalar FlowMonitor
    ejecutar hasta T = 200 s
    guardar el XML de FlowMonitor y el resumen de métricas
```

4.4 Diseño experimental

Para responder la pregunta se comparan cuatro escenarios de movilidad: E0, sin movimiento; E1, con solo movilidad de nodo; E2, con solo movilidad de clúster; y E3, con ambas capas. Los escenarios con movimiento se evalúan a 1, 5, 10 y 20 m/s, y cada configuración se repite diez veces con semillas distintas, para un total de 130 corridas. De cada corrida se obtienen la tasa de entrega, el retardo, el *throughput* y el *jitter* a partir de los flujos de datos registrados por FlowMonitor. El análisis compara las medias con intervalos de confianza del 95 % y contrasta E2 contra E1 mediante una prueba t de Welch unilateral.

La comparación entre E1 y E2 es la que responde la pregunta de investigación. Como en ambos se mueve una sola capa a la misma velocidad, la diferencia observada en la tasa de entrega y en el retardo se puede atribuir al tipo de movilidad y no a un cambio en la cantidad de movimiento. El escenario E3, que combina ambas capas, permite comprobar si los efectos se acumulan cuando el movimiento ocurre en los dos niveles a la vez.

4.5 Decisiones de diseño y su justificación

Varias decisiones merecen justificarse explícitamente porque condicionan la interpretación de los resultados. Se usaron cuatro clústeres en el nivel inferior, y no tres, para que los dos superclústeres quedaran simétricos y los flujos pudieran repartirse de forma equilibrada entre tráfico intra e inter-superclúster. Los superclústeres se mantienen fijos para que, en cada escenario, se mueva una sola capa a la velocidad de estudio; de otro modo no sería posible atribuir el efecto observado a un tipo de movilidad concreto. La propagación se modela con un disco de alcance exacto, y no con el modelo log-distancia por defecto de ns-3, porque con la potencia y el umbral de detección estándar un enlace 802.11b a 1 Mbps solo alcanza unos 50 m, insuficiente para sostener un *backbone* entre clústeres separados 150 m; el modelo de disco, además, hace que la geometría sea fácil de controlar y de explicar. El tráfico se distribuyó en ocho flujos de baja carga, cuatro dentro de cada superclúster y cuatro entre superclústeres, para forzar el uso de la jerarquía sin saturar la red. Y se descartó ns3-ai

porque el problema no requiere aprendizaje ni control adaptativo: la medición del efecto de la movilidad sobre AODV se resuelve de forma natural con la simulación de eventos discretos de ns-3, y prescindir de capas adicionales reduce el riesgo de incompatibilidades y acelera las corridas.

5 Código fuente

El programa de simulación está implementado en C++ sobre ns-3 (versión 3.43), en el archivo `simulacion/manet-jerarquica.cc`. Esta sección recorre el entorno de software utilizado, la organización del código fuente, la generación de tráfico, la captura de métricas con `FlowMonitor`, la visualización en NetAnim y la validación sobre el escenario base. El objetivo es que cualquier lector pueda ubicar en el código cada pieza del diseño descrito en la sección 4.

5.1 Entorno de software y versiones

El desarrollo y las simulaciones se hicieron en el entorno de la tabla 6, acordado por el equipo para que los scripts y los resultados sean directamente reproducibles. Fijar las versiones no es un detalle menor: un cambio de versión de ns-3 puede alterar el comportamiento de los modelos y, con ello, los resultados.

Componente	**Versión**	**Observaciones / Rol en el proyecto**
Simulador	ns-3.43	Versión acordada por el grupo, siguiendo las guías del curso.
Sistema operativo de referencia	Ubuntu 22.04 LTS	Entorno documentado en el plan; nativo o bajo WSL2 en Windows.
Compilador C++	GCC / g++ 11.4.0	Estándar ISO C++17 activado por defecto.
Gestor de compilación	CMake 3.22.1 / Ninja	Invocado a través del script `./ns3`.
Lenguaje de análisis	Python 3.10+	Bibliotecas `pandas`, `matplotlib` y `scipy` (fases 4 y 7).
Visualizador gráfico	NetAnim 3.109	Opcional para inspección visual de topología y trayectorias.

Cuadro 6: Plataforma y versiones de software utilizadas en la implementación.

Sobre la biblioteca `ns3-ai`: Aunque el enunciado menciona `ns3-ai` como una alternativa deseable, se optó por no utilizarla. El estudio aborda modelos analíticos de movilidad y el comportamiento del protocolo reactivo AODV, fenómenos que el motor de simulación de eventos discretos de ns-3 resuelve de forma nativa en C++. Incorporar `ns3-ai`, en cambio, exigía capas de sincronización entre procesos mediante memoria compartida y dependencias de Python que complicaban la instalación y aumentaban el tiempo de cómputo, sin aportar al análisis de la movilidad. Mantener la simulación en C++ nativo simplifica el despliegue y asegura reproducibilidad exacta con el generador de números pseudoaleatorios (`--RngRun`). En pocas palabras, la herramienta se eligió por lo que exige el problema y no por lo que ofrece la librería.

5.2 Estructura de `manet-jerarquica.cc`

El programa está organizado de forma modular, de modo que el trabajo de cada integrante quedara aislado en sus propias funciones y no hubiera dependencias cruzadas en el código:

- `struct Parametros`: agrupa las opciones configurables desde la línea de comandos mediante la clase `CommandLine` de ns-3 (escenario, velocidades, tiempos, alcances de radio y ruta de salida).

- `struct RedJerarquica`: estructura de datos compartida que almacena los `NodeContainer` organizados por niveles (todos los nodos, clústeres A–D, superclústeres X e Y, lista de Cluster Heads y miembros), así como los contenedores de interfaces IP correspondientes.
- **Red y direccionamiento** (`CrearNodos`, `ConfigurarRed`): responsabilidad de la Parte 1. Instancia los 20 nodos, configura canales WiFi independientes por clúster con modelo de disco (`RangePropagationLossModel`), canal compartido para el backbone entre CH, y asigna la pila de red con AODV y direccionamiento por subredes.
- **Movilidad jerárquica** (`ConfigurarMovilidad`): responsabilidad de la Parte 2. Conecta las tres capas relativas de movimiento mediante modelos anidados.
- **Tráfico y métricas** (`ConfigurarTrafico`, `ConfigurarMetricas`, `GuardarMetricas`, `ConfigurarNetAnim`): responsabilidad de la Parte 3. Inyecta los flujos de prueba, instala FlowMonitor, serializa los archivos XML e imprime el resumen en consola.

El punto de entrada (`main`) orquesta las etapas en secuencia: procesa parámetros, crea la topología, aplica movilidad, instala tráfico y monitores, ejecuta el simulador y exporta las métricas al finalizar.

5.3 Generación de tráfico sintético (`ConfigurarTrafico`)

Para poner a prueba la jerarquía completa de la red se definen ocho flujos UDP de tasa constante (CBR) mediante aplicaciones `OnOffApplication`. Todos los flujos van entre nodos miembros de clústeres diferentes y combinan comunicación dentro de un mismo superclúster con comunicación que cruza entre superclústeres:

- **Flujos intra-superclúster (miembros 1):** $A1 \leftrightarrow B1$ dentro del superclúster X, y $C1 \leftrightarrow D1$ dentro del superclúster Y (puertos 5000 a 5003).
- **Flujos inter-superclúster (miembros 2):** $A2 \rightarrow C2$, $B2 \rightarrow D2$, $C2 \rightarrow A2$ y $D2 \rightarrow B2$, cruzando la frontera entre X e Y (puertos 5004 a 5007).

Todos los generadores y receptores están en nodos miembros. Los Cluster Heads (nodo 0 de cada clúster) no emiten ni reciben tráfico de usuario: actúan solo como enrutadores intermedios en el backbone. Así se evita que un tráfico propio del CH compita en sus colas con el tráfico de tránsito, que es justamente el que se quiere medir.

Cada flujo emite paquetes de 512 bytes a 4 paquetes por segundo, es decir, una tasa fija de 16 384 bit/s (16 kbit/s). La aplicación se configura con `OnTime` constante de 1.0 y `OffTime` en 0.0, lo que produce una emisión continua y periódica. El inicio de cada flujo se desfasa con una variable uniforme en el intervalo $[10.0, 11.0]$ segundos, para que los flujos no comiencen todos a la vez y saturen el medio con solicitudes de ruta simultáneas. Esa ventana inicial de 10 segundos permite, además, que los módulos inalámbricos se sincronicen y que AODV descubra las rutas de forma ordenada. La transmisión se detiene a los 195 segundos, lo que deja un margen de 5 segundos para que los paquetes en vuelo o en búferes de reintento lleguen a su destino antes del fin de la simulación.

5.4 Recolección y reporte de métricas (`FlowMonitor`)

Generar trazas completas de paquetes (PCAP o ASCII) para 130 ejecuciones produciría un volumen de datos excesivo y, en su mayoría, innecesario. Por eso se emplea el módulo `FlowMonitor` de ns-3, que monitorea en memoria cada flujo IPv4 identificado por su tupla de cinco elementos (origen, destino, puertos y protocolo) y entrega directamente los contadores que necesitan las métricas.

Al concluir la simulación, `GuardarMetricas()` procesa las estadísticas agregadas de todos los flujos y calcula las métricas del experimento:

- **Tasa de entrega (PDR):** porcentaje de paquetes recibidos respecto a los transmitidos ($\mathrm{PDR} = 100 \cdot \sum \mathrm{rxPackets} / \sum \mathrm{txPackets}$).
- **Retardo promedio extremo a extremo:** cociente entre la suma de retardos de paquetes recibidos y el total de paquetes recibidos ($\sum \mathrm{delaySum} / \sum \mathrm{rxPackets}$).
- **Throughput agregado:** tasa en kbit/s calculada a partir de los bytes recibidos sobre la ventana activa de tráfico ($8 \cdot \sum \mathrm{rxBytes} / (T_{\mathrm{activo}} \cdot 1000)$).
- **Jitter medio:** variación promedio del retardo entre paquetes consecutivos recibidos ($\sum \mathrm{jitterSum} / \sum (\mathrm{rxPackets} - 1)$).

El reporte completo de FlowMonitor se exporta en formato XML en la ruta acordada:

`<outDir>/<scenario>_cs<clusterSpeed>_ns<nodeSpeed>_run<RngRun>.xml`

La función auxiliar `NombreSalida()` centraliza este formato, y `std::filesystem::create_directories` asegura que el directorio de salida exista antes de la escritura. Para facilitar la inspección durante las pruebas manuales, el programa imprime además un resumen compacto de las métricas en la terminal. Ese resumen suma todos los flujos que registra FlowMonitor, incluidos los mensajes de control de AODV, por lo que puede diferir ligeramente de las métricas del análisis, que usan solo los ocho flujos de datos.

5.5 Soporte para NetAnim (`ConfigurarNetAnim`)

Con el argumento `--netanim=true`, el programa genera un archivo de animación XML en el directorio de salida. Para distinguir los roles de la topología, los Cluster Heads se dibujan en rojo, con un tamaño ampliado (radio 6.0) y la etiqueta `[CH]`, mientras que los miembros aparecen en azul y con tamaño estándar (radio 3.5). Esta diferenciación permite comprobar a simple vista la coherencia espacial de los clústeres y su movimiento relativo durante la depuración.

5.6 Fragmentos clave de la implementación

A continuación se incluyen los fragmentos centrales de la Parte 3 en `simulacion/manet-jerarquica.cc`.

5.6.1 Configuración de flujos CBR

```
const std::vector<Flujo> flujos = {
    {0, 1, 1, 1}, // A1 -> B1 (intra X)
    {1, 1, 0, 1}, // B1 -> A1 (intra X)
    {2, 1, 3, 1}, // C1 -> D1 (intra Y)
    {3, 1, 2, 1}, // D1 -> C1 (intra Y)
    {0, 2, 2, 2}, // A2 -> C2 (inter X-Y)
    {1, 2, 3, 2}, // B2 -> D2 (inter X-Y)
    {2, 2, 0, 2}, // C2 -> A2 (inter X-Y)
    {3, 2, 1, 2}  // D2 -> B2 (inter X-Y)
};

Ptr<UniformRandomVariable> inicio = CreateObject<UniformRandomVariable>();
inicio->SetAttribute("Min", DoubleValue(INICIO_TRAFICO_MIN));
inicio->SetAttribute("Max", DoubleValue(INICIO_TRAFICO_MAX));

for (uint32_t f = 0; f < flujos.size(); ++f) {
    const Flujo& fl = flujos[f];
```

```
    uint16_t puerto = puertoBase + f;
    Ipv4Address ipDestino = red.ifCluster[fl.clusterDestino].GetAddress(fl.miembroDestino);

    PacketSinkHelper sumidero("ns3::UdpSocketFactory",
                              InetSocketAddress(Ipv4Address::GetAny(), puerto));
    ApplicationContainer appRx =
        sumidero.Install(red.clusters[fl.clusterDestino].Get(fl.miembroDestino));
    appRx.Start(Seconds(0.0));
    appRx.Stop(Seconds(p.simTime));

    OnOffHelper emisor("ns3::UdpSocketFactory", InetSocketAddress(ipDestino, puerto));
    emisor.SetAttribute("PacketSize", UintegerValue(512));
    emisor.SetAttribute("DataRate", DataRateValue(DataRate("16384bps")));
    emisor.SetAttribute("OnTime", StringValue("ns3::ConstantRandomVariable[Constant=1]"));
    emisor.SetAttribute("OffTime", StringValue("ns3::ConstantRandomVariable[Constant=0]"));

    ApplicationContainer appTx =
        emisor.Install(red.clusters[fl.clusterOrigen].Get(fl.miembroOrigen));
    appTx.Start(Seconds(inicio->GetValue()));
    appTx.Stop(Seconds(fin));
}
```

5.6.2 Cálculo y guardado de métricas

```
std::filesystem::create_directories(p.outDir);
std::string archivo = NombreSalida(p, "");

g_monitor->CheckForLostPackets();
g_monitor->SerializeToXmlFile(archivo, true, true);

// Acumulación de estadísticas por flujo
for (const auto& [id, st] : g_monitor->GetFlowStats()) {
    tx += st.txPackets;
    rx += st.rxPackets;
    rxBytes += st.rxBytes;
    retardo += st.delaySum.GetSeconds();
    jitter += st.jitterSum.GetSeconds();
    if (st.rxPackets > 1) {
        rxParaJitter += st.rxPackets - 1;
    }
}
```

5.7 Validación de integración del programa

Antes de automatizar los experimentos, conviene comprobar que el sistema completo funciona en el caso más sencillo. Por eso se valida sobre el escenario de control **E0**, sin movilidad:

```
./ns3 run "scratch/manet-jerarquica --scenario=E0 --clusterSpeed=0 \
          --nodeSpeed=0 --simTime=200 --RngRun=1 --outDir=resultados/raw"
```

En la réplica 1 (`E0_cs0_ns0_run1.xml`), los ocho flujos transmitieron un total de 5 901 paquetes de datos y se recibieron exactamente 5 901, alcanzando una tasa de entrega (PDR) del 100.0 %, un retardo promedio de 18.82 ms y un throughput agregado de 137.80 kbit/s. A través de las diez réplicas de control ejecutadas por el equipo (`resultados/resumen.csv`), la

PDR media se situó en el 99.97 % (con un intervalo de confianza al 95 % de $[99.93, 100.01]$ %) y el retardo promedio en 20.41 ms.

Estos resultados confirman que la red fija es estable y que el enrutamiento AODV a través del backbone jerárquico opera sin pérdidas una vez descubiertas las rutas iniciales. Queda así validada la línea base: si los escenarios E1, E2 y E3 muestran degradación, esta podrá atribuirse a la movilidad y no a defectos en la pila de red o en la generación de tráfico.

6 Manual de usuario y manual técnico

El manual técnico y el manual de usuario se entregan en un documento aparte, `Manuales.pdf`. El manual técnico describe los requisitos, la instalación de ns-3, la arquitectura del programa y cómo extenderlo; el manual de usuario explica, paso a paso, cómo correr una simulación, ejecutar todos los escenarios, generar las gráficas e interpretar la salida. Ambos se generan desde `Manuales.tex` y comparten las fuentes de este informe.

7 Experimentación y análisis de resultados

7.1 Escenarios y ejecución

Esta sección pone a prueba la hipótesis con los datos. Se ejecutaron los cuatro escenarios de la tabla 3 con los parámetros de la tabla 4: E0 (sin movilidad), E1 (solo movilidad de nodo), E2 (solo movilidad de clúster) y E3 (ambas). E1, E2 y E3 se corrieron a las velocidades $v \in \{1, 5, 10, 20\}$ m/s y E0 una sola vez, cada configuración con diez réplicas (`RngRun` $= 1, \ldots, 10$): 130 corridas de 200 s en total, todas completadas sin errores y todas con paquetes recibidos. Las corridas se automatizaron con `scripts/correr_escenarios.sh`, las métricas se extrajeron de los XML de FlowMonitor con `scripts/parse_flowmon.py` y las tablas y figuras se generaron con `scripts/graficar.py`.

Que un programa corra sin errores no garantiza que haga lo que se diseñó. Por eso, antes del experimento, el programa se verificó con una batería de 46 pruebas automáticas (`scripts/validar_movilidad.py`). Las pruebas confirmaron que en E0 ningún nodo se mueve, que en E1 los CH permanecen fijos, que en E2 cada miembro conserva exactamente su posición relativa a su CH, que clústeres y nodos no salen de sus regiones (± 150 m y ± 40 m), que nunca se supera la velocidad configurada, que la misma semilla reproduce resultados idénticos y que en E0 se entregan todos los paquetes. Las corridas son además reproducibles entre computadores: la réplica 1 de E2 a 20 m/s dio la misma PDR (65.06 %) en la validación y en el experimento.

Tratamiento de los datos. Las métricas se calculan solo sobre los 8 flujos de datos (puertos UDP 5000–5007). FlowMonitor registra además los mensajes de control de AODV (puerto UDP 654), que se excluyen porque no son tráfico de la aplicación y su volumen cambia con la movilidad. Incluirlos sesgaría la comparación entre escenarios, porque se estaría midiendo, en parte, el esfuerzo de la red por reorganizarse y no el servicio que presta. Los intervalos de confianza del 95 % usan la distribución t de Student con $n - 1 = 9$ grados de libertad (sección 2.6).

7.2 Resultados agregados

La tabla 7 resume la media y el intervalo de confianza del 95 % de cada métrica por escenario y velocidad.

7.3 Interpretación por escenario

E0 (control). Sin movilidad la red entrega el 99.97 % de los paquetes (mínimo 99.8 % entre réplicas), con un retardo de unos 20 ms. La red está conectada y la línea base es estable,

Esc.	v (m/s)	PDR (%)	Retardo (ms)	Throughput (kbit/s)	Jitter (ms)
E0	—	99.97 ± 0.04	20.4 ± 1.6	137.7 ± 0.1	1.37 ± 0.81
E1	1	95.67 ± 4.86	19.2 ± 1.1	131.8 ± 6.7	1.02 ± 0.69
E1	5	97.78 ± 2.41	19.4 ± 1.2	134.7 ± 3.3	1.17 ± 0.75
E1	10	97.08 ± 3.51	19.4 ± 1.3	133.7 ± 4.8	1.24 ± 0.86
E1	20	96.39 ± 2.66	19.4 ± 1.1	132.8 ± 3.7	1.19 ± 0.71
E2	1	79.55 ± 8.43	25.4 ± 10.7	109.6 ± 11.6	2.57 ± 2.52
E2	5	75.21 ± 7.29	30.2 ± 11.2	103.6 ± 10.1	5.09 ± 2.69
E2	10	70.39 ± 3.95	53.2 ± 23.0	97.0 ± 5.4	10.20 ± 4.10
E2	20	70.51 ± 5.08	93.7 ± 21.1	97.2 ± 7.0	21.05 ± 4.30
E3	1	83.09 ± 9.87	20.4 ± 3.2	114.5 ± 13.6	1.70 ± 1.09
E3	5	76.90 ± 5.89	44.2 ± 18.1	105.9 ± 8.1	8.18 ± 3.95
E3	10	72.08 ± 6.48	64.2 ± 18.4	99.3 ± 8.9	12.54 ± 3.78
E3	20	71.82 ± 5.37	100.9 ± 31.0	98.9 ± 7.4	23.83 ± 6.54

Cuadro 7: Media ± semiancho del intervalo de confianza del 95 % ($n = 10$ réplicas por fila). Fuente: `resultados/tabla_estadistica.csv`.

como exige el diseño (PDR $\geq$ 95 %). Las pocas pérdidas corresponden al descubrimiento inicial de rutas de AODV.

E1 (solo nodos). La movilidad de nodo apenas afecta el desempeño: la PDR se mantiene entre 95.7 % y 97.8 % y el retardo en unos 19 ms, prácticamente igual al control, a cualquier velocidad. Esto se explica por la geometría: un miembro se mueve como máximo ±40 m en cada eje alrededor de su CH, es decir, a lo sumo a $40\sqrt{2} \approx 56.6$ m, dentro del alcance de 60 m del clúster. Los enlaces del backbone no cambian y el miembro casi siempre tiene un enlace directo con su CH. Las pérdidas que aparecen (algunas réplicas bajan hasta 78.8 %) se deben a rutas locales de AODV que pasan por otro miembro y se rompen cuando este se aleja.

E2 (solo clústeres). El panorama cambia por completo cuando se mueven los clústeres. La movilidad de clúster produce la mayor degradación: la PDR cae a entre 70 % y 80 % y el retardo crece de 25 ms a 94 ms al pasar de 1 a 20 m/s. Al desplazarse hasta ±150 m de su superclúster, dos CH pueden quedar a más de 250 m y el backbone se fragmenta: los paquetes entre clústeres se pierden o esperan en cola mientras AODV busca una ruta nueva. Esa actividad de reparación se refleja en el número de flujos de control de AODV que registra FlowMonitor, que sube de 17 en E0 a 21 a 1 m/s y a 55 a 20 m/s, mientras que en E1 se mantiene en 18–19. La red, en otras palabras, dedica cada vez más esfuerzo a reconstruir rutas y entrega cada vez menos.

E3 (ambas). El escenario combinado se comporta como E2: sus medias de PDR (71.8 %–83.1 %) y de retardo (20 ms–101 ms) no difieren significativamente de las de E2 a ninguna velocidad (prueba t de Welch bilateral, $p > 0.15$ en las ocho comparaciones). Sumar la movilidad de nodo a la de clúster no añade una degradación apreciable.

7.4 Comparación E1 vs. E2: prueba de la hipótesis principal

La tabla 8 presenta las pruebas t de Welch unilaterales [19] de la hipótesis principal.

En PDR se rechaza H_0 en las cuatro velocidades, con diferencias de 16 a 27 puntos porcentuales. En retardo se rechaza en tres de las cuatro: a 1 m/s el retardo de E2 es mayor (25.4 frente a 19.2 ms), pero su alta variabilidad entre réplicas impide que la diferencia sea significativa. Según la regla de decisión definida de antemano (al menos tres de cuatro velocidades), **la hipótesis principal queda respaldada para ambas métricas**.

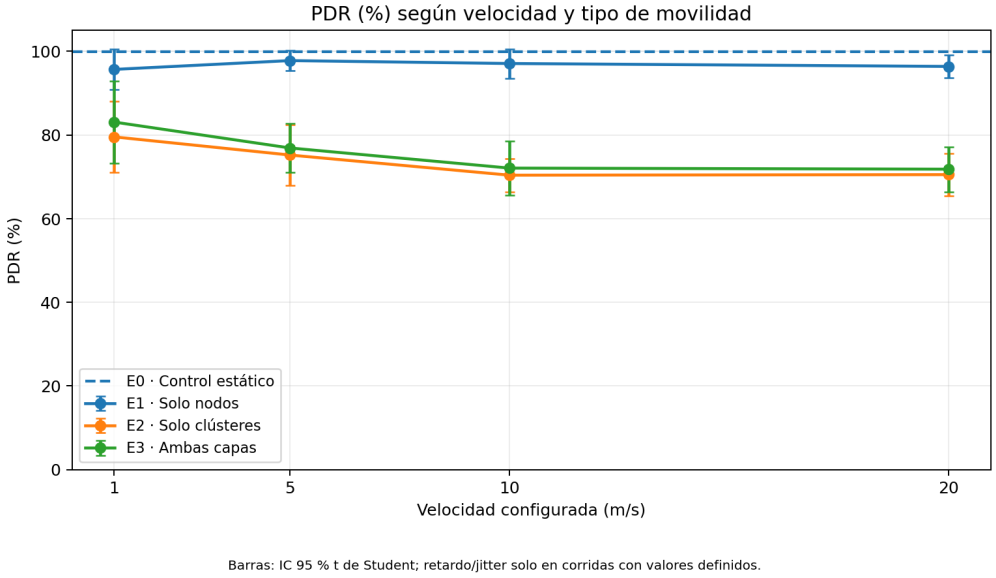


Figura 4: PDR media frente a la velocidad, con intervalos de confianza del 95 %. La línea discontinua es el control E0.

v	**Métrica**	**E1**	**E2**	t	p	**¿Rechaza H_0?**
1	PDR (%)	95.67	79.55	−3.75	0.0010	Sí
5	PDR (%)	97.78	75.21	−6.65	1.8×10^{-5}	Sí
10	PDR (%)	97.08	70.39	−11.42	$< 10^{-8}$	Sí
20	PDR (%)	96.39	70.51	−10.21	5×10^{-8}	Sí
1	Retardo (ms)	19.2	25.4	1.31	0.111	No
5	Retardo (ms)	19.4	30.2	2.16	0.029	Sí
10	Retardo (ms)	19.4	53.2	3.33	0.0044	Sí
20	Retardo (ms)	19.4	93.7	7.96	1.1×10^{-5}	Sí

Cuadro 8: Prueba t de Welch unilateral E2 vs. E1 por velocidad ($\alpha = 0.05$, $n = 10$ por grupo). H_1: PDR(E2) < PDR(E1) y retardo(E2) > retardo(E1). Fuente: `resultados/comparacion_E2_vs_E1.csv`.

7.5 Efecto de la velocidad e hipótesis secundarias

HS1 (el desempeño empeora con la velocidad). Se cumple solo cuando hay movilidad de clúster. En E2 y E3 la PDR a 20 m/s es significativamente menor que a 1 m/s ($p = 0.028$ y $p = 0.020$, Welch unilateral) y el retardo se multiplica por 3.7 y 4.9 ($p < 0.001$), con una correlación de Spearman entre velocidad y retardo de 0.76 y 0.81. En E1, en cambio, ni la PDR ni el retardo cambian con la velocidad ($\rho \approx 0$, $p > 0.4$). HS1 queda **parcialmente respaldada**.

Cabe notar que en E2 y E3 la PDR deja de bajar entre 10 y 20 m/s (70.4 % frente a 70.5 % en E2), mientras que el retardo y el jitter siguen creciendo. Una explicación es que la fracción del tiempo en que el backbone está partido depende sobre todo de la geometría (dónde están los clústeres), no de la velocidad; la velocidad cambia, en cambio, la frecuencia con que se rompen y reconstruyen las rutas, que es lo que aumenta el retardo. A esto se suma una propiedad conocida de Random Waypoint [20]: como la pausa de 2 s es fija, a mayor velocidad los clústeres pasan más tiempo detenidos (cerca del 4 % del tiempo a 1 m/s y del 21 % a 20 m/s, medido en la validación), por lo que su velocidad media efectiva es menor que la configurada.

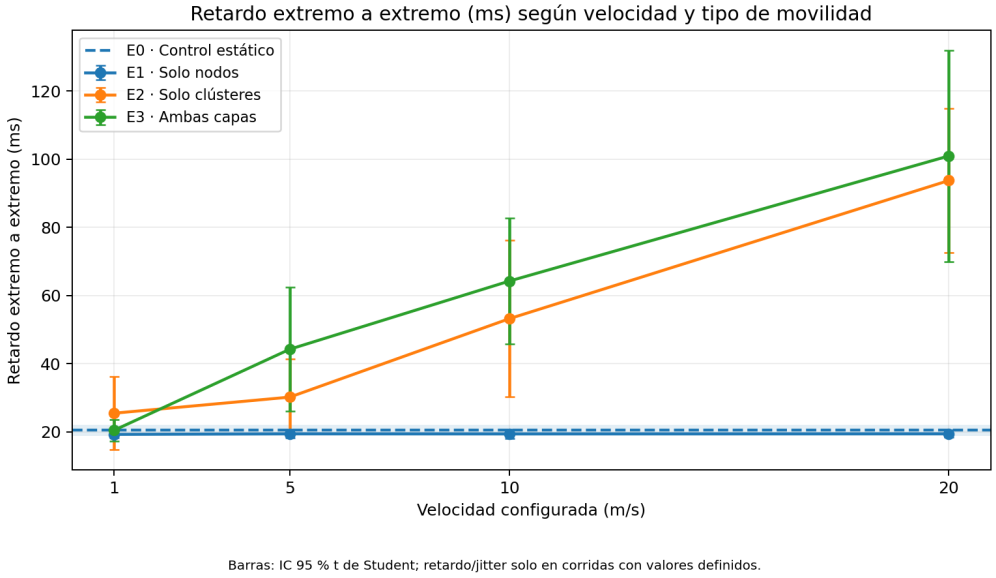


Figura 5: Retardo extremo a extremo medio frente a la velocidad, con intervalos de confianza del 95 %.

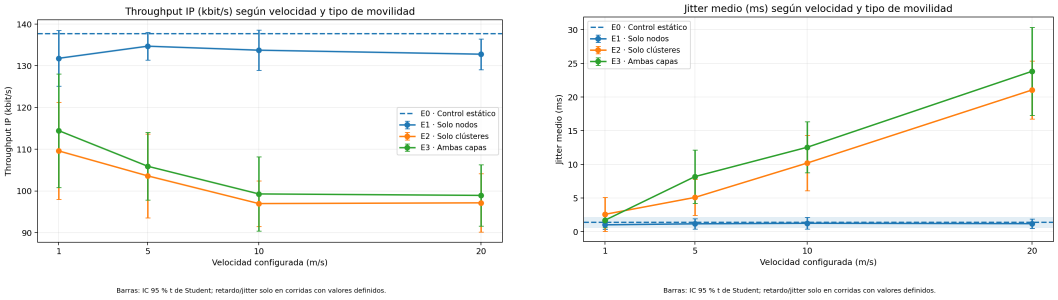


Figura 6: Throughput agregado (izquierda) y jitter medio (derecha) frente a la velocidad.

HS2 (E3 igual o peor que el peor de E1 y E2). El peor escenario individual es siempre E2, y E3 no difiere significativamente de él en ninguna velocidad. HS2 **se respalda en su forma "igual"**: el efecto combinado no es aditivo y está dominado por la movilidad de clúster.

HS3 (la diferencia entre E1 y E2 crece con la velocidad). La diferencia de retardo crece de forma monótona (6.2, 10.8, 33.8 y 74.3 ms a 1, 5, 10 y 20 m/s). La diferencia de PDR crece de 16.1 a 26.7 puntos entre 1 y 10 m/s y luego se estabiliza (25.9 puntos a 20 m/s). HS3 queda **respaldada para el retardo y parcialmente para la PDR**.

8 Conclusiones

Las 130 corridas permiten responder la pregunta planteada en la sección 2 con evidencia estadística. Las conclusiones se resumen a continuación.

1. **Respuesta a la pregunta de investigación.** En la MANET jerárquica de dos niveles simulada, con enrutamiento AODV, la movilidad a nivel de clúster degrada el desempeño mucho más que la movilidad a nivel de nodo. A igual velocidad, el escenario con solo movilidad de clúster (E2) entrega entre 16 y 27 puntos porcentuales menos

paquetes que el escenario con solo movilidad de nodo (E1), y su retardo es entre 1.3 y 4.8 veces mayor. Mover el grupo completo resulta, en otras palabras, mucho más dañino que dejar que cada integrante se mueva por su cuenta.

2. **Se acepta la hipótesis principal.** La prueba t de Welch unilateral rechaza H_0 para la PDR en las cuatro velocidades ($p \leq 0.002$) y para el retardo en tres de las cuatro (5, 10 y 20 m/s). Ambas métricas cumplen la regla de decisión fijada antes del experimento (al menos tres de cuatro velocidades). Los resultados son coherentes con el mecanismo propuesto: la movilidad de clúster rompe los enlaces entre CH, por donde pasa todo el tráfico entre clústeres, mientras que la movilidad de nodo solo altera rutas locales.

3. **El efecto de la velocidad depende del tipo de movilidad (HS1, parcialmente respaldada).** Con movilidad de clúster, mayor velocidad implica menor PDR y, sobre todo, mucho más retardo y jitter (el retardo pasa de unos 25 ms a 94–101 ms entre 1 y 20 m/s). Con solo movilidad de nodo, la velocidad no tiene efecto medible, porque en este diseño un miembro nunca sale del alcance de su CH.

4. **La movilidad de clúster domina el efecto combinado (HS2).** El escenario con ambas movilidades (E3) no es significativamente distinto de E2 en ninguna velocidad: la degradación no se suma, sino que la determina el nivel superior de la jerarquía. Una vez que el backbone se rompe, el movimiento adicional de los nodos apenas cambia el resultado.

5. **La brecha entre ambos tipos de movilidad crece con la velocidad (HS3).** La diferencia de retardo entre E2 y E1 crece de forma monótona (de 6 ms a 74 ms) y la de PDR crece hasta 10 m/s y luego se estabiliza en unos 26 puntos.

6. **Implicación práctica.** En redes jerárquicas, la estabilidad de la columna vertebral entre cluster heads es el factor crítico del desempeño. Los esfuerzos de diseño (elección de CH, alcance del backbone, rutas de respaldo entre clústeres) rinden más que proteger los enlaces dentro de cada clúster.

9 Recomendaciones

9.1 Limitaciones del estudio

Los resultados anteriores son sólidos dentro del escenario simulado, pero ese escenario tiene límites que conviene hacer explícitos antes de extrapolarlos:

- **Velocidad efectiva de Random Waypoint.** Los clústeres hacen una pausa fija de 2 s en cada destino, y a mayor velocidad pasan una fracción mayor del tiempo detenidos (cerca del 4 % a 1 m/s y del 21 % a 20 m/s). La velocidad media efectiva es, por tanto, menor que la nominal, un problema conocido de este modelo [20, 11].

- **Geometría que protege a E1.** Con regiones de ± 40 m y alcance de 60 m, un miembro siempre queda al alcance directo de su CH. Esto explica la robustez de E1, pero también significa que la conclusión depende de la relación entre el tamaño de la región de los nodos y el alcance del clúster. Con regiones más grandes, la movilidad de nodo podría pesar más.

- **Modelo de propagación idealizado.** El modelo de disco no representa desvanecimiento, sombras ni interferencia gradual: un enlace existe por completo o no existe.

- **Escala reducida.** Se usaron 20 nodos, 4 clústeres, 8 flujos de baja tasa (16 kbit/s), CH fijos dentro de su clúster y un solo protocolo (AODV). Los resultados no deben generalizarse a cualquier MANET sin nuevos experimentos.

- **Comparaciones múltiples.** Las pruebas de hipótesis se hicieron por velocidad con $\alpha = 0.05$ sin ajuste por comparaciones múltiples. Para la PDR esto no cambia la conclusión ($p \leq 0.002$ en todas las velocidades), pero la prueba de retardo a 5 m/s ($p = 0.029$) no superaría una corrección de Bonferroni.
- **Diez réplicas por configuración.** Los intervalos de confianza de E2 y E3 son amplios (por ejemplo, ± 21 ms en el retardo de E2 a 20 m/s). Más réplicas permitirían detectar diferencias menores, como la de retardo a 1 m/s.

9.2 Trabajo futuro

Cada una de estas limitaciones sugiere, a su vez, una línea para continuar el estudio:

- Usar modelos de movilidad sin el sesgo de velocidad de Random Waypoint, como Gauss-Markov o Random Direction, y un modelo de grupo como RPGM [7, 5].
- Variar el tamaño de la región de los nodos respecto al alcance del clúster, para encontrar el punto en que la movilidad de nodo empieza a degradar la red.
- Comparar AODV con protocolos proactivos como OLSR [6] o DSDV, que reaccionan de forma distinta a la ruptura del backbone.
- Medir la sobrecarga de enrutamiento (paquetes de control de AODV por paquete de datos entregado), que en los datos ya disponibles crece claramente con la movilidad de clúster.
- Escalar a más nodos y niveles jerárquicos, mover también los superclústeres (`--superSpeed`) y permitir la reelección dinámica de CH.
- Explorar ns3-ai para adaptar en línea parámetros de enrutamiento o la elección de CH según la movilidad observada.

Referencias

[1] Jerry Banks, John S. Carson, Barry L. Nelson, and David M. Nicol. *Discrete-Event System Simulation*. Pearson, 5 edition, 2010.

[2] Michel Barbeau and Evangelos Kranakis. *Principles of Ad Hoc Networking*. John Wiley & Sons, 2007. Archivo del curso: MANETS/Redes Ad Hoc (1).pdf.

[3] Josh Broch, David A. Maltz, David B. Johnson, Yih-Chun Hu, and Jorjeta Jetcheva. A performance comparison of multi-hop wireless ad hoc network routing protocols. In *Proceedings of the 4th Annual ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom)*, pages 85–97, 1998.

[4] Mario Bunge. *La ciencia: su método y su filosofía*. 1995.

[5] Tracy Camp, Jeff Boleng, and Vanessa Davies. A survey of mobility models for ad hoc network research. *Wireless Communications and Mobile Computing*, 2(5):483–502, 2002.

[6] Thomas Clausen and Philippe Jacquet. Optimized link state routing protocol (OLSR). RFC 3626, Internet Engineering Task Force, 2003.

[7] Xiaoyan Hong, Mario Gerla, Guangyu Pei, and Ching-Chuan Chiang. A group mobility model for ad hoc wireless networks. In *Proceedings of the 2nd ACM International Workshop on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, pages 53–60, 1999.

[8] Pierre L'Ecuyer. Good parameters and implementations for combined multiple recursive random number generators. *Operations Research*, 47(1):159–164, 1999.

[9] Pierre L'Ecuyer, Richard Simard, E. Jack Chen, and W. David Kelton. An object-oriented random-number package with many long streams and substreams. *Operations Research*, 50(6):1073–1075, 2002.

[10] Ben Liang and Zygmunt J. Haas. Predictive distance-based mobility management for PCS networks. In *Proceedings of IEEE INFOCOM '99*, volume 3, pages 1377–1384, 1999.

[11] William Navidi and Tracy Camp. Stationary distributions for the random waypoint mobility model. *IEEE Transactions on Mobile Computing*, 3(1):99–108, 2004.

[12] Juan Pablo Ospina-López and Jorge Eduardo Ortiz-Triviño. Estimation of a growth factor to achieve scalable ad hoc networks. *Ingeniería y Universidad*, 21(1):49–70, 2017.

[13] Charles E. Perkins, Elizabeth M. Belding-Royer, and Samir R. Das. Ad hoc on-demand distance vector (AODV) routing. RFC 3561, Internet Engineering Task Force, 2003.

[14] Subir Kumar Sarkar, T. G. Basavaraju, and C. Puttamadappa. *Ad Hoc Mobile Wireless Networks: Principles, Protocols, and Applications*. CRC Press, 2 edition, 2013.

[15] The ns-3 Project. Flow monitor. ns-3 Model Library, `https://www.nsnam.org/docs/models/html/flow-monitor.html`. Consultado el 4 de noviembre de 2026.

[16] The ns-3 Project. Mobility. ns-3 Model Library, `https://www.nsnam.org/docs/models/html/mobility.html`. Consultado el 4 de noviembre de 2026.

[17] The ns-3 Project. Random variables. ns-3 Manual, `https://www.nsnam.org/docs/manual/html/random-variables.html`. Consultado el 4 de noviembre de 2026.

[18] The ns-3 Project. ns-3 network simulator: Documentation. `https://www.nsnam.org/documentation/`, 2024.

[19] B. L. Welch. The generalization of "Student's" problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, 1947.

[20] Jungkeun Yoon, Mingyan Liu, and Brian Noble. Random waypoint considered harmful. In *Proceedings of IEEE INFOCOM 2003*, volume 2, pages 1312–1321, 2003.

[21] Henry Zárate Ceballos, Jorge Ernesto Parra Amaris, Hernán Jiménez Jiménez, Diego Alexis Romero Rincón, Oscar Agudelo Rojas, and Jorge Eduardo Ortiz Triviño. *Wireless Network Simulation: A Guide using Ad Hoc Networks and the ns-3 Simulator*. Apress, 2021.